PUBLIC REPOSITORY

product-catalog

A robust full-stack Product Catalog Application built using modern web development methodologies. The architecture guarantees a clean separation of concerns between client interface workflows and core administrative backend services.

🏗️ Project Architecture

The system is configured as a cleanly separated monorepo split into localized frontend and backend modules:

```
product-catalog/
├── front/               # Client Interface Module (Vite + React)
│   ├── .env             # UI environment configuration profiles
│   ├── index.html       # Single Page Application core template
│   └── package.json     # Production bundles & web development utilities
└── back/                # Data Core Engine & Services Infrastructure
    ├── node_modules/    # Vendor ecosystems (Express framework, query engines)
    └── package.json     # Structural backend application manifest
```

Key Tech Stacks

- **Frontend Sub-module:** Structured using a rapid development build system powered by the `Vite Engine` and managed dynamically via `React Router DOM` infrastructure. Client-to-server data serialization and networking processes are orchestrated securely via `Axios`.
- **Backend Sub-module:** Built with robust schema and request enforcement using `Express Validator` blocks combined with standard routing layers and object model query filters (`mquery` / `sift` core utilities).

🛠️ Environmental Settings

Configure local variables inside the frontend context directory before initializing build processes.

Client Configuration (`front/.env`)

Create an environment initialization block inside your local configuration script:

```
VITE_API_USERNAME=admin
VITE_API_PASSWORD=secret123
```

🚀 Setup and Initial Execution

Prerequisites

Ensure that an active Node.js environment layer along with the `npm` packet execution utility is configured globally on your developer system.

System Initialization

1. Clone and Navigate

```
git clone <repository-url>
cd product-catalog
```

2. Core Backend Engine Construction

Enter the server-side runtime directory boundaries to bootstrap module dependencies and run the server instance:

```
cd back
npm install
npm start
```

3. Client Interface Deployment

Instantiate framework assets needed for frontend modules and run the Vite environment pipeline locally:

```
cd ../front
npm install
npm run dev
```

🧪 Verified Workflows and Tooling

- **Bundler & Minification Engines:** Handled by `esbuild` system architecture runtimes alongside automated `Rollup` code-splitting utilities for high-performance frontend asset compilation.
- **Testing Implementations:** Continuous processing check routines integrated via localized automated script ecosystems to evaluate request stream reliability.